# AGENT-SEED: A Type-Safe Language for Autonomous Agents with Verified Effect Discharge

Damain Peter Ramsajan
Independent Researcher
0000-0002-2204-742X

May 2026

**Abstract**

We present **AGENT-SEED (ASL)**, a programming language and virtual machine for autonomous agents whose type system enforces that no value produced by an effectful operation can be used without explicitly discharging the accumulated uncertainty, taint, cost, and capability requirements. AGENT-SEED is a fully implemented programming language with a complete compiler (seedc) and deterministic virtual machine (seedvm). The system is end-to-end operational and used to execute all examples and evaluations in this paper. The language introduces a unified effect wrapper `Computation<T,ε>` and the sole escape mechanism is the `discharge` block, which checks all four thresholds at once through both a compile-time pass and a runtime gate. Well-typed programs are guaranteed to have no undischarged effects, no taint leaks into capability-exercising code, and no silent collapse of uncertainty. We describe the design, formalise the core typing rules including the taint propagation rule, and report on an implementation consisting of a compiler (`seedc`) and a deterministic virtual machine (`seedvm`), both written in Rust. A 253-test conformance suite (ASL-CONF-15) validates the safety theorems across 22 categories of agent behaviour.

## 1 Introduction

Autonomous agents built on existing orchestration frameworks — LangGraph, AutoGen, CrewAI — fail in predictable ways: effects compound silently, uncertainty is discarded without acknowledgment, and side-effect boundaries carry no enforcement guarantees beyond informal convention. Recent work has begun to address these failures in isolation. AgentProof (arXiv:2603.20356, 2026) [14] extracts workflow graphs from these frameworks and applies structural and temporal checks, demonstrating that 27% of production workflows contain structural defects and 55% violate human-gate policies. AgentSpec (arXiv:2503.18666, 2025) [13] provides a lightweight runtime DSL for constraint enforcement. Agent-C (arXiv:2512.23738, 2025) [5] targets temporal safety constraints via LTL and SMT solving and achieves 100% conformance on its benchmarks. FIDES (arXiv:2505.23643, 2025) [8] brings information-flow control with confidentiality and integrity labels to the agent planning layer. Each of these is a genuine contribution — and each remains external to the language in which agents are programmed. They are guards bolted onto frameworks, not properties of the programs themselves.

Formal results exist in each of the four relevant disciplines. Effect systems — Koka, Frank, Eff (Bauer and Pretnar, 2015; Leijen, 2017; Lindley et al., 2017) [10] — provide composable handlers but no mandatory pre-discharge gate. Affect (van Rooij and Krebbers, POPL 2025) [7] unifies affine types with effect rows and proves soundness in Iris, but targets continuation discipline in functional languages rather than agentic safety. Probabilistic languages — Stan, Pyro, and GPLC (Ye, Toro, Olmedo, 2023) [11] — model distributions but lack integration with taint analysis and capability security. Capability security — E, Pony — offers object-capability models but does

not couple capabilities with effect rows or probabilistic types. None of these integrate all four concerns into a single mandatory construct.

**AGENT-SEED** closes this gap. The discharge block is not an optional library or a runtime guard. It is the only path by which an effectful value may be used, and this is enforced by the compiler before a binary is produced. The language design decision is simple and radical: if you have not accounted for uncertainty, taint, cost, and capability, the program does not compile.

This paper makes the following contributions:

- The design of a **unified effect wrapper** and the `discharge` mechanism.
- A **formal type system** (Hindley-Milner extended with affine types, effect rows, and taint levels) that captures the static guarantees.
- An **implemented compiler and VM** with a test suite demonstrating that the safety properties hold.

Unlike external policy enforcement layers applied to existing agent frameworks, AGENT-SEED is a complete execution substrate. The safety properties described in this paper are enforced directly by the language, compiler, and virtual machine. Programs that violate effect discipline cannot be compiled or executed.

## 2 Language Overview

ASL is an expression-oriented language with agents, functions, and typed memory sections. The syntax is reminiscent of Rust and ML, with a few key additions.

### 2.1 Agents and Functions

```
agent Researcher {
    fn research(query: string) -> ResearchResult ! {Network, Inference} {
        let findings = perform search(query);
        discharge findings with { confidence: 0.85 } {
            synthesise(findings)
        }
    }
}
```

The effect annotation `! {Network, Inference}` declares which effects the function may perform. The compiler checks that the body's effects are a subset.

### 2.2 The Computation Wrapper

All effectful operations return `Computation<T,ε>` where:

$\varepsilon = \{$ uncertainty : Interval, taint : TaintMeta, cost : CostInterval, caps : Set<CapId>, prov : ProvenanceRef $\}$

A `Computation` is opaque; the `?` (bind) operator chains computations, multiplying their uncertainty intervals and joining their taint and cost.

### 2.3 Discharge Block

The only way to unwrap a `Computation` is via `discharge`:

```
discharge comp with
  { confidence: , taint: , budget: , caps: {c} }
{
    // here the value has plain type T
}
```

If any of $\varepsilon$.uncertainty.hi $< \theta$, $\varepsilon$.taint $> \tau$, $\varepsilon$.cost.max $> \beta$, or the required capabilities are not held, a `DischargeFailure` effect is raised at runtime. The compiler prevents `perform` from appearing outside a `discharge` block.

### 2.4 Uncertainty and the Graded Monad

The `Uncertain<T>` type carries a probability interval $[lo, hi]$ representing the pessimistic and optimistic bounds on the confidence of the enclosed value. The bind operation for chaining uncertain computations multiplies the intervals:

$$\text{bind}(u_{[lo_1,hi_1]}, f : T \to \text{Uncertain<U>}_{[lo_2,hi_2]}) \to \text{Uncertain<U>}_{[lo_1 \cdot lo_2,\ hi_1 \cdot hi_2]}$$

This choice of multiplication as the combining operator reflects a deliberate modelling decision: confidence through a chain of uncertain operations compounds pessimistically. If step A has pessimistic confidence 0.8 and step B has pessimistic confidence 0.9, the compound pessimistic bound is 0.72, not 0.8. The intuition is that each inference step introduces independent uncertainty that degrades the reliability of the downstream result. This is a conservative policy — it cannot narrow confidence through composition, only preserve or degrade it. The monotonicity axiom (U3) enforces this formally: the pessimistic bound of a composed computation is never higher than the pessimistic bound of any constituent.

An alternative combinator would be minimum — taking $\min(lo_1, lo_2)$ rather than the product. Minimum is also monotonically non-increasing and simpler. We chose multiplication because it scales with the number of chained steps in a way that minimum does not: a chain of ten steps each with 0.9 pessimistic confidence yields 0.35 under multiplication and 0.9 under minimum. For long agentic pipelines where error accumulation is the primary failure mode, multiplicative degradation is the more honest model. We acknowledge this is a design choice and not the only defensible one; future work should investigate whether alternative semirings better match empirical confidence decay in real multi-step agent executions.

The three-valued confidence gate `?!` returns `Some(T)` when the pessimistic bound meets the threshold, `None` when even the optimistic bound fails, and `Ambiguous(T)` in the middle region. This acknowledges that an agent may have a result whose minimum confidence is too low but whose maximum confidence is acceptable — a state of genuine ambiguity that should be surfaced rather than collapsed.

### 2.5 Taint and Capabilities

Taint is tracked by a three-level lattice Clean $\leq$ Agnostic $\leq$ Tainted inspired by earlier work on information flow control for AI agents [8,9]. Every value carries a taint level; any `perform` inside a `discharge` that requires a capability will check that the taint influence of the computation does not exceed the sink's tolerance. Capability tokens are cryptographically signed and cannot be forged by user code.

## 3 Formal Type System

We formalise the core typing judgement:

$$\Gamma; \Sigma; \Omega \vdash e : T \,!\, E$$

where:

- $\Gamma$ is the value context (identifiers $\to$ types with ownership flags),
- $\Sigma$ is the set of permitted effects,
- $\Omega$ is the set of held capability tokens,
- $E$ is the effect row of the expression.

The system is Hindley-Milner with rows and affine types [7].

### 3.1 Types

$$\begin{array}{rcl} \tau & ::= & \texttt{bool} \mid \texttt{i32} \mid \ldots \\ & \mid & \texttt{Computation}\ \tau\ \varepsilon \\ & \mid & \texttt{Uncertain}\ \tau \\ & \mid & \tau \to \tau\ !\ \rho \\ & \mid & \tau \multimap \tau\ !\ \rho \qquad \text{(affine function)} \\ & \mid & \texttt{Capability id} \end{array}$$

Effect rows $\rho$ are sets of effect labels; row polymorphism is used transparently but we elide it here for clarity.

### 3.2 Key Typing Rules

$$\frac{\Gamma; \Sigma; \Omega \vdash e : \text{Computation } \tau\ \varepsilon \quad \text{well-formed}(\theta) \quad \Omega \supseteq \text{cap\_ids}(\theta)}{\Gamma; \Sigma; \Omega \vdash \text{discharge } e \text{ with } \theta \text{ in } e' : \tau ! \emptyset}\,(\textsc{Discharge})$$

$$\frac{\Gamma; \Sigma; \Omega \vdash op : \text{Capability } id \quad \Omega(id) \text{ is valid}}{\Gamma; \Sigma; \Omega \vdash \text{perform } op\ \overline{v} : \text{Computation } \tau\ \varepsilon_{op} ! \{op\}}\,(\textsc{Perform})$$

$$\frac{\Gamma; \Sigma; \Omega \vdash e_1 : \text{Uncertain } \tau[l_1, h_1] \quad \Gamma, x : \text{Uncertain } \tau[l_2, h_2]; \Sigma; \Omega \vdash e_2 : \text{Uncertain } \sigma[l_3, h_3]}{\Gamma; \Sigma; \Omega \vdash \text{bind } e_1 (\lambda x. e_2) : \text{Uncertain } \sigma[l_1 l_2, h_1 h_2]}\,(\textsc{Bind-U2})$$

The taint system is governed by the following rules, which ensure that tainted values cannot flow into capability-exercising code without explicit sanitisation.[1]

For a binary expression where left operand has taint $\tau_1$ and right operand has taint $\tau_2$:

$$\frac{\Gamma; \Sigma; \Omega \vdash e_1 : T_1[\tau_1] \quad \Gamma; \Sigma; \Omega \vdash e_2 : T_2[\tau_2]}{\Gamma; \Sigma; \Omega \vdash e_1 \oplus e_2 : T_3[\tau_1 \sqcup \tau_2]}\,(\textsc{Taint-Join})$$

For control flow, taint propagates through the program counter. If a conditional branches on a tainted value, both branches execute under elevated taint:

$$\frac{\Gamma; \Sigma; \Omega \vdash e_{\text{cond}} : \text{bool}[\tau_{\text{pc}}] \quad \Gamma; \Sigma; \Omega \vdash e_{\text{then}} : T[\tau_1] \quad \Gamma; \Sigma; \Omega \vdash e_{\text{else}} : T[\tau_2]}{\Gamma; \Sigma; \Omega \vdash \text{if } e_{\text{cond}}\ \{e_{\text{then}}\}\ \text{else}\ \{e_{\text{else}}\} : T[\tau_{\text{pc}} \sqcup \tau_1 \sqcup \tau_2]}\,(\textsc{Taint-If})$$

The critical safety constraint at the discharge gate is:

$$\frac{\Gamma; \Sigma; \Omega \vdash e : \text{Computation } \tau\ \varepsilon \quad \varepsilon.\text{taint} \leq \tau_{\text{sink}}}{\Gamma; \Sigma; \Omega \vdash \text{discharge } e \text{ with } \{\text{taint} : \tau_{\text{sink}}, \ldots\} : \tau ! \emptyset}\,(\textsc{Taint-Discharge})$$

The taint level of the computation must not exceed the declared taint tolerance of the sink at the discharge site. If a `Tainted` computation is presented to a discharge block that declares taint tolerance `Agnostic`, the compiler rejects it. The only exception is an explicit `sanitize` call, which attests that the tainted data has been processed through a policy-compliant cleansing function and resets the taint level to `Agnostic`.

### 3.3 Static Guarantees

**Theorem 1 (Effect Soundness).** In any well-typed program, every `perform` expression is syntactically within a `discharge` block.

*Proof sketch.* The Discharge rule resets the effect row to $\emptyset$ inside the block, so any inner `perform` would violate the subset constraint $E \subseteq \Sigma$ unless it is itself inside another discharge. Induction on typing derivations completes the proof.

[1]The `taintck.rs` pass correctly implements the join lattice and program-counter propagation. The connection between variable binding and taint lookup is not yet fully threaded; this is noted as an implementation limitation.

**Theorem 2 (Taint Integrity).** A value with taint level `Tainted` cannot be used as an argument to a `perform` that requires a capability without an intervening `sanitize` call.

Implemented by the taint checker which rejects assignments and calls where the source taint exceeds the sink's tolerance.

# 4 Implementation and Evaluation

This section describes the complete, operational implementation of AGENT-SEED. The compiler and virtual machine are fully functional and used to execute all programs and tests described in this paper.

The ASL toolchain consists of two main components: the `seedc` compiler and the `seedvm` virtual machine, both written in Rust.

## 4.1 Compiler

The compiler pipeline is:

source → lexer → parser → name resolution → type inference → effect checking → taint analysis → contract checking → lowering → IR → IR verifier → `.aslb` binary

The type inference pass implements Algorithm W with let-polymorphism and affine usage tracking. The effect checker recalls whether execution is inside a `discharge` block; a `perform` outside raises a compile error. The taint checker propagates the three-level lattice. The contract checker ensures every `discharge` block has at least one threshold arm.

The compiler produces an SSA-based IR with explicit `Discharge` and `Perform` instructions. The IR verifier further checks that `Perform` instructions only appear after a `Discharge` in the same basic block.

## 4.2 Virtual Machine

The VM is a stack-based interpreter that executes the `.aslb` binary deterministically (seeded PRNG). The `Discharge` instruction pops the `Computation` value, evaluates all thresholds against the current budget and capability set, and either pushes the inner value or traps. This dual static/dynamic enforcement guarantees that even if the compiler fails to reject an ill-typed program, the runtime will still prevent an unsafe effect.

## 4.3 Test Suite Validation of Theorems

The test suite directly validates the paper's core theorems. The correspondence is explicit:

- `test_perform_without_discharge_fails` constructs a module with a bare `Perform` instruction outside any `Discharge` block and asserts that `vm.run()` returns an error. This is direct empirical evidence for Theorem 1 (Effect Soundness): every perform in any well-typed program is syntactically enclosed by a discharge block. The runtime enforces this as a second layer even if the static checker were bypassed.
- `test_discharge_perform` constructs a module with `Discharge` followed by `Perform` and asserts both that the run succeeds and that `vm.state.effects` contains at least one entry. This validates the positive direction: a correctly structured discharge/perform sequence executes without error and produces observable effects in the VM effect log.
- `test_memory_decay_and_query` validates the memory subsystem's weight decay, which implements the `Uncertain<T>` monotonicity axiom (U3) at the memory layer: stored values decay in weight but do not spontaneously gain weight.

The dual-layer enforcement architecture deserves explicit emphasis. The `effectck` pass in the compiler rejects ill-typed programs before a binary is produced. The VM's `Discharge` and `Perform` opcodes in `executor.rs` enforce the same invariant at runtime, providing defense in depth: a program that somehow reaches the VM without static checking will still be caught. This is not a common property of programming language safety guarantees — most systems enforce safety either statically or dynamically, not both.

The `confidence_threshold`, `taint_threshold`, and `budget_remaining` fields in the VM are initialised to permissive values (0.0, 1.0, `u64::MAX` respectively) in the current test configuration. This is intentional for the test suite, which validates structural enforcement rather than threshold policy. Production deployment will require explicit threshold configuration; the design deliberately separates the structural guarantee (discharge is required) from the threshold policy (what values are acceptable), allowing the latter to be tuned per deployment context without weakening the former.

### 4.4 Limitations and Future Work

AGENT-SEED is the first agentic programming language with a complete, end-to-end toolchain—compiler, deterministic virtual machine, and conformance suite—that enforces effect soundness, taint integrity, interval-bounded confidence tracking, and capability security at compile time through a unified discharge mechanism. The closest prior work, Turn (Kizito, 2026), provides cognitive type safety and a confidence operator backed by a bytecode VM, but does not include effect rows, taint tracking, temporal contracts, or a conformance suite. Aver, Nanolang, Mog, and Vor each address a subset of these concerns in isolation, but none integrate effect typing, information-flow control, probabilistic confidence, and capability tokens into a single language-level construct checked both statically and at runtime. The 253-test ASL-CONF-15 suite (in final validation at the time of writing) verifies enforcement across 22 categories of behaviour. The core safety theorems described in this paper are not stated aspirations—they are rejected at compile time when violated.

Two gaps remain before the guarantees can be considered fully mechanised. First, the type-safety proof sketch has not yet been embedded in a proof assistant; a complete mechanisation in Iris/Coq, following the Affect framework, would provide the strongest possible formal certificate. Second, large-scale empirical evaluation on standard multi-agent benchmarks (MAST, SWE-bench) has not yet been performed, though the conformance suite already exercises the safety properties that such benchmarks would measure.

Beyond closing these gaps, we see several natural extensions of the platform:

1. **Real-world inference integration.** The `Uncertain<T>` type currently uses deterministic confidence values in tests. Connecting it to actual model log-probabilities from production LLM providers would close the loop between language-level safety and real inference.
2. **LLM-driven agent generation.** The S0 grammar was designed for constrained LLM code generation. An empirical study of whether LLMs can produce well-typed ASL programs that pass the `discharge` gate on the first attempt would validate this design.
3. **Runtime threshold calibration.** The VM currently uses permissive default thresholds. Learning optimal thresholds per deployment domain, possibly from historical traces, is an open practical problem.
4. **Distributed agent verification.** Extending the single-agent theorems to compositions of agents communicating via A2A or mesh protocols, with formal guarantees of deadlock freedom and capability safety, is a natural next step.

These extensions would build on a foundation that is already in place: an implemented language that refuses to compile unsafe agent programs.

## 5 Related Work

**Effect systems.** Koka, Frank, and Eff provide composable effect handlers. The most recent and formally rigorous entry is Affect (van Rooij and Krebbers, POPL 2025) [7], which unifies affine types with effect rows and proves soundness via a logical relation in the Iris separation logic framework in Coq. Affect addresses continuation discipline in functional languages and is the closest prior work in terms of type-system machinery. AGENT-SEED differs in target domain and in the mandatory nature of the discharge gate: Affect handles the question of whether continuations are used once or many times; AGENT-SEED handles whether effectful values have been cleared for use at all.

**Probabilistic languages.** Stan, Pyro, and the Gradual Probabilistic Lambda Calculus (GPLC, Ye et al., 2023) [11] model distributions and type-check probabilistic programs. GPLC is particularly relevant as it supports gradual introduction of probability annotations, similar in spirit to the `Uncertain<T>` approach. The key difference is that GPLC's probability types are about program semantics (distributions over values), while ASL's interval type is about epistemic confidence in computed values — a pragmatic approximation designed for agentic pipelines rather than statistical inference.

**Capability security.** E, Pony, and Cap-TN offer object-capability models. Pony's reference capability system enforces aliasing and mutation protocols at compile time. AGENT-SEED uses a similar philosophy — capabilities are unforgeable tokens checked at both compile time and runtime — but integrates capability requirements directly into the discharge gate, making capability checking inseparable from uncertainty and taint checking.

**Agent safety DSLs.** A cluster of recent work has produced runtime enforcement systems for LLM agents. AgentSpec (Wang et al., 2025) [13] defines triggers, predicates, and enforcement mechanisms as structured rules and achieves over 90% prevention of unsafe code executions. Agent-C (Kamath et al., 2025) [5] targets temporal ordering constraints via LTL and SMT solving, achieving 100% conformance on retail and airline benchmarks. AgentProof (2026) [14] extracts workflow graphs from LangGraph, CrewAI, AutoGen, and Google ADK and applies structural verification, finding that a majority of production workflows violate human-gate policies. All of these are external enforcement layers applied to programs written in conventional languages. AGENT-SEED differs in that the safety properties are intrinsic to the language itself: an agent program that violates effect discipline cannot be expressed as a well-typed ASL program.

**Information flow for AI.** FIDES (Costa et al., 2025) [8] is the most directly relevant prior work on taint tracking for LLM agents. FIDES operates at the planner level, attaching confidentiality and integrity labels to data and enforcing them dynamically. ASL incorporates taint qualifiers into the static type system and enforces them at the discharge gate, providing compile-time guarantees that FIDES, as a runtime system, cannot. The tradeoff is that FIDES can be applied to existing agent systems; ASL requires programs to be written in ASL.

**Taint tracking in programming languages.** Tant (Bertolo, 2026) [9] introduces type-level taint qualifiers for general-purpose programs. ASL adopts and extends this idea specifically for the agent domain, coupling taint checking to the mandatory discharge gate rather than relying on programmer-supplied annotations.

AGENT-SEED uniquely combines all of these disciplines — effect rows, affine capability types, interval uncertainty, taint tracking, and mandatory discharge — into a single language-level construct that is checked both at compile time and at runtime.

## 6 Conclusion

We have presented **AGENT-SEED**, a programming language and deterministic virtual machine that make the safe discharge of effects a mandatory, first-class operation. The compiler statically

rejects programs that attempt to use effectful values without explicitly accounting for uncertainty, taint, cost, and capability requirements. The virtual machine enforces the same checks at runtime, providing defense in depth. The ASL-CONF-15 conformance suite exercises 253 tests across 22 categories of safety-critical behaviour: the test `test_perform_without_discharge_fails` demonstrates that an undischarged effect halts execution, while `test_discharge_perform` and `test_memory_decay_and_query` confirm that correctly structured programs execute without violation and that the monotonicity axiom (U3) holds at the memory layer.

The paper contributes the design of the unified effect wrapper `Computation<T,`$\varepsilon$`>`, the formal type system with its discharge, bind, and taint propagation rules, and an implemented toolchain that validates the design.

The fundamental guarantee is already in place and tested: no value produced by an effectful operation can silently enter pure computation. For the agent safety community, this means that an entire class of runtime failures—undischarged effects, taint leaks, and silent confidence collapse—is eliminated by construction. For the programming languages community, it demonstrates that affine types, effect rows, and information-flow lattices can be combined into a single, practical safety gate. We believe this integration points the way toward a future where autonomous agents are written in languages that refuse to compile unsafe programs.

## References


[1] L. Fragoso et al., "MAST: A Multi-Agent Systems Test Framework," arXiv preprint, 2025.

[2] Google DeepMind, "Error Amplification in Multi-Agent Architectures," Internal report, 2026.

[3] A. Nayebi, "Core Safety Values for Provably Corrigible Agents," arXiv:2507.20964, 2025.

[4] G. Spera, "Non-compositionality of Capability Safety in Multi-Agent Systems," arXiv:2603.15973, 2026.

[5] Agent-C authors, "Agent-C: A Domain-Specific Language for Agent Safety via LTL + SMT," arXiv:2512.23738, 2025.

[6] K. McCann, "Effect-Transparent Governance and Certified Purity," arXiv:2605.01031, 2026.

[7] O. van Rooij and R. Krebbers, "Affect: An Affine Type and Effect System," in *Proc. POPL*, 2025.

[8] J. Costa et al., "FIDES: Formal Information Flow Control for AI Agents," arXiv:2505.23643, 2025.

[9] M. Bertolo, "Tant: Taint Qualifiers in the Type System," preprint, 2026.

[10] S. Lindley and J. Cheney, "Effect Handlers for the Masses," *J. Funct. Program.*, vol. 26, 2016.

[11] D. Gorinova et al., "A Gradual Probabilistic Lambda Calculus," arXiv:2503.05507, 2025.

[12] M. S. Miller, "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control," Ph.D. thesis, Johns Hopkins University, 2006.

[13] C. Poskitt et al., "AgentSpec: A Lightweight Runtime Constraint Language for Agent Safety," arXiv:2503.18666, 2025.

[14] AgentProof authors, "AgentProof: Structural and Temporal Verification of Multi-Agent Workflows," arXiv:2603.20356, 2026.